

Project Design Phase-II Technology Stack (Architecture & Stack)

Date	29 September 2026
Team ID	SWTID-2026-8776
Project Name	AI WeatherWise API
Maximum Marks	4 Marks

Chevalier T. Thomas Elizabeth College for Women

Team:

Sairunisha S M – Team Leader

Swetha R

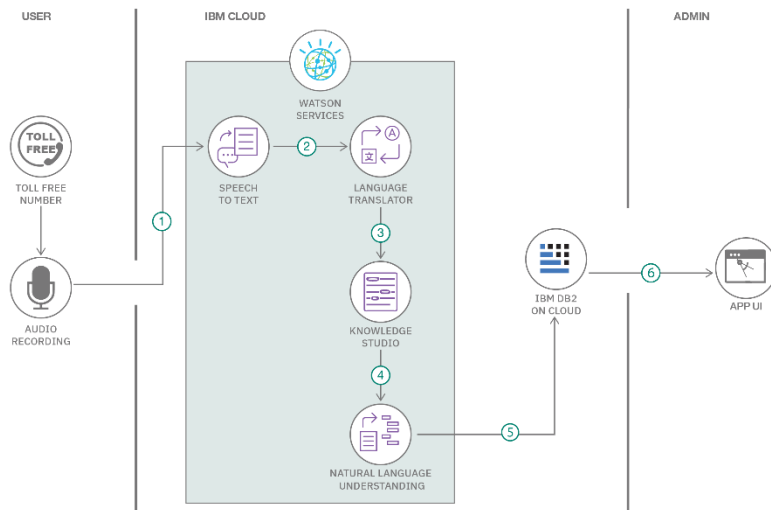
Lokeshwari P

Yuvashri R

Technical Architecture:

AI WeatherWise API follows a **layered RESTful backend architecture**. The client sends requests to the Express.js server, where authentication middleware validates JWT tokens before protected operations are processed by controllers. The controllers communicate with MongoDB through Mongoose and with external services such as **OpenWeather** and **Google Gemini AI**. The processed information is returned to the client as JSON. **Example: Order processing during pandemics for offline mode**

Reference: <https://developer.ibm.com/patterns/ai-powered-backend-system-for-order-processing-during-pandemics/>



Guidelines:

- Include all the processes (As an application logic / Technology Block)
- Provide infrastructural demarcation (Local / Cloud)
- Indicate external interfaces (third party API's etc.)
- Indicate Data Storage components / services
- Indicate interface to machine learning models (if applicable)

Table-1 : Components & Technologies:

S.No.	Component	Description	Technology
1	User Interface / API Client	Interface used to send requests and view JSON responses.	Thunder Client / Postman
2	Application Logic – Server	Handles HTTP requests, routing, middleware, CORS, and server operations.	Node.js, Express.js
3	Application Logic – Authentication	Registers users, validates credentials, hashes passwords, and protects routes.	bcryptjs, JWT
4	Application Logic – Weather & AI	Retrieves weather data and generates AI summaries and recommendations.	Node.js, Express.js, Google Gemini AI
5	Database	Stores user accounts and favorite locations.	MongoDB, Mongoose ODM
6	Cloud Database	Cloud database option for deployment.	MongoDB Atlas
7	File Storage	No dedicated file-storage service is required by the current backend.	Local filesystem / Not applicable
8	External API – 1	Provides real-time weather information such as temperature, humidity, wind, and conditions.	OpenWeather API

9	External API – 2	Generates natural-language weather summaries and personalized recommendations.	Google Gemini API
10	Machine Learning / AI Model	Processes weather information and generates natural-language insights.	Google Gemini AI
11	Infrastructure	Backend can be executed locally using Node.js; deployment can later use cloud infrastructure.	Local Node.js Server / Cloud deployment

Table-2: Application Characteristics:

S.No.	Characteristics	Description	Technology
1	Open-Source Frameworks	Uses open-source backend frameworks and libraries for API development and database interaction.	Node.js, Express.js, Mongoose
2	Security Implementations	Passwords are hashed before storage. JWT authentication protects secured endpoints. CORS and request validation help control API access.	bcryptjs, JWT, CORS, Express middleware
3	Scalable Architecture	Modular REST API architecture separates routing, authentication, controllers, models, database access, and external services. This allows individual components to be extended without redesigning the entire system.	Node.js, Express.js, MongoDB, Mongoose
4	Availability	The API can run continuously as a Node.js server and can be deployed to cloud infrastructure for wider availability. The external weather and AI services are accessed through APIs.	Node.js, Cloud Deployment*
5	Performance	RESTful endpoints return structured JSON responses. Mongoose provides efficient database operations, while external API calls are performed only when weather or AI information is requested.	Express.js, Mongoose, OpenWeather API, Gemini API

AI WeatherWise API uses a modular and scalable backend architecture built with **Node.js and Express.js**, with **MongoDB/Mongoose** for data persistence, **JWT and bcryptjs** for security, **OpenWeather API** for real-time weather information, and **Google Gemini AI** for intelligent weather summaries and personalized recommendations. The architecture separates authentication, business logic, database operations, and external service integrations, making the system maintainable and extensible.

References:

<https://c4model.com/>

<https://developer.ibm.com/patterns/online-order-processing-system-during-pandemic/>

<https://www.ibm.com/cloud/architecture>

<https://aws.amazon.com/architecture>

<https://medium.com/the-internal-startup/how-to-draw-useful-technical-architecture-diagrams-2d20c9fda90d>